The Course Project

The course project includes 3 parts. The first part is to develop a Python MRJob application to retrieve Year and Temperature from original NCDC records (i.e., the dataset we are using for this class) and then write the Year and Temperature data into a text file. The second part is to load the text file into Pig and get the highest and lowest temperatures for each year. The third part is to load the text file into Hive and get the average temperature for each year. You need to turn in

1) Python MRJob coding file,

2) the commands for executing the Python MRJob application in Hadoop,

3) the text file including Year and Temperature data created by you,

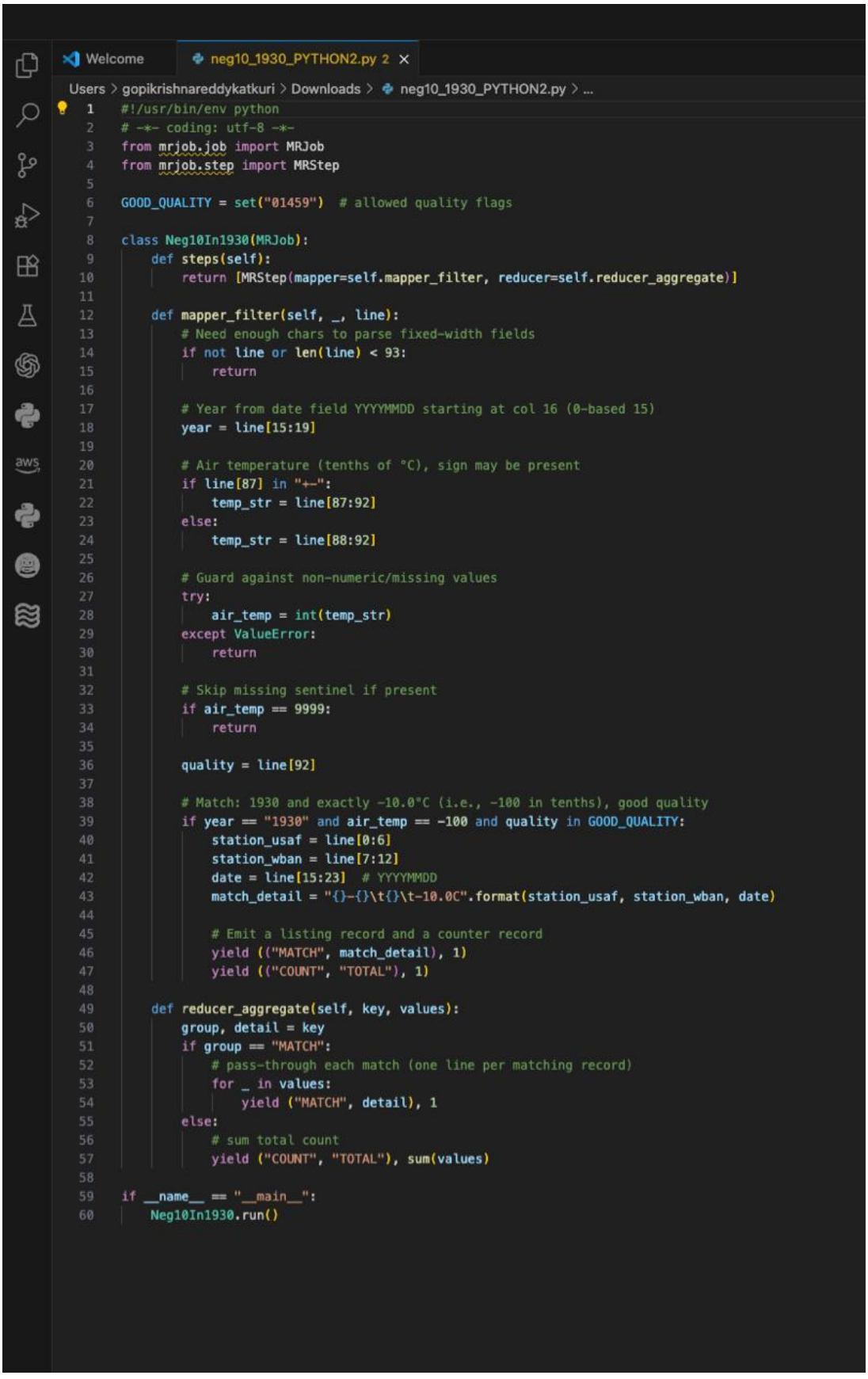
4) the screenshot of the text file being created,

5) the screenshot of the final Pig output showing the year and the highest and lowest temperatures, and

6) the screenshot of the final Hive output showing the year and average temperature. The original dataset for this project is available on Canvas.

Step 1: Develop a Python Mrjob Application

TempExtractor_Python2.py

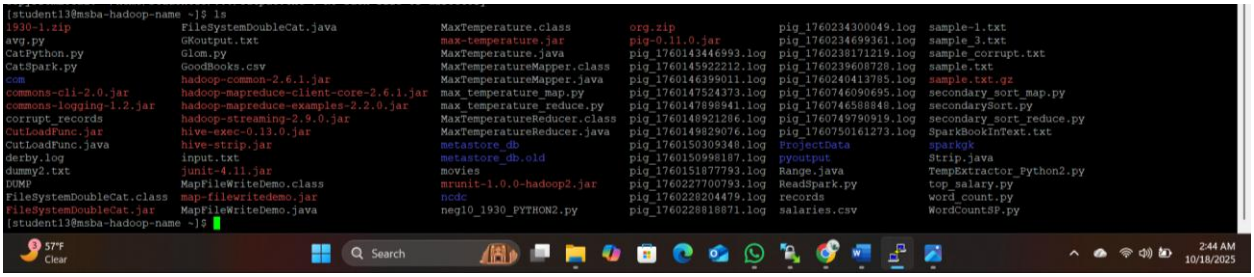

Welcome
neg10_1930_PYTHON2.py 2
Users > gopikrishnareddykatkuri > Downloads > neg10_1930_PYTHON2.py > ...
#!/usr/bin/env python
-*- coding: utf-8 -*-
from mrjob.job import MRJob
from mrjob.step import MRStep
GOOD_QUALITY = set("01459") # allowed quality flags
class Neg10In1930(MRJob):
def steps(self):
return [MRStep(mapper=self.mapper_filter, reducer=self.reducer_aggregate)]
def mapper_filter(self, _, line):
Need enough chars to parse fixed-width fields
if not line or len(line) < 93:
return
Year from date field YYYYMMDD starting at col 16 (0-based 15)
year = line[15:19]
Air temperature (tenths of °C), sign may be present
if line[87] in "+-":
temp_str = line[87:92]
else:
temp_str = line[88:92]
Guard against non-numeric/missing values
try:
air_temp = int(temp_str)
except ValueError:
return
Skip missing sentinel if present
if air_temp == 9999:
return
quality = line[92]
Match: 1930 and exactly -10.0°C (i.e., -100 in tenths), good quality
if year == "1930" and air_temp == -100 and quality in GOOD_QUALITY:
station_usaf = line[0:6]
station_wban = line[7:12]
date = line[15:23] # YYYYMMDD
match_detail = "{}-{}\t{}\t-10.0C".format(station_usaf, station_wban, date)
Emit a listing record and a counter record
yield (("MATCH", match_detail), 1)
yield (("COUNT", "TOTAL"), 1)
def reducer_aggregate(self, key, values):
group, detail = key
if group == "MATCH":
pass-through each match (one line per matching record)
for _ in values:
yield ("MATCH", detail), 1
else:
sum total count
yield ("COUNT", "TOTAL"), sum(values)
if __name__ == "__main__":
Neg10In1930.run()

Make folders and transfer necessary files (as required in filesystem and hdfs) –

Transfer files to local filesystem:

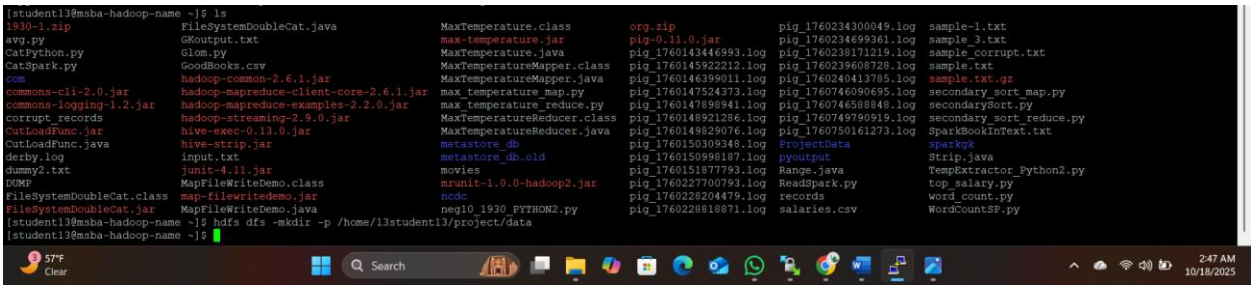
1. TempExtractor_Python2.py

2. ProjectData.zip (present inside the ProjectData folder)



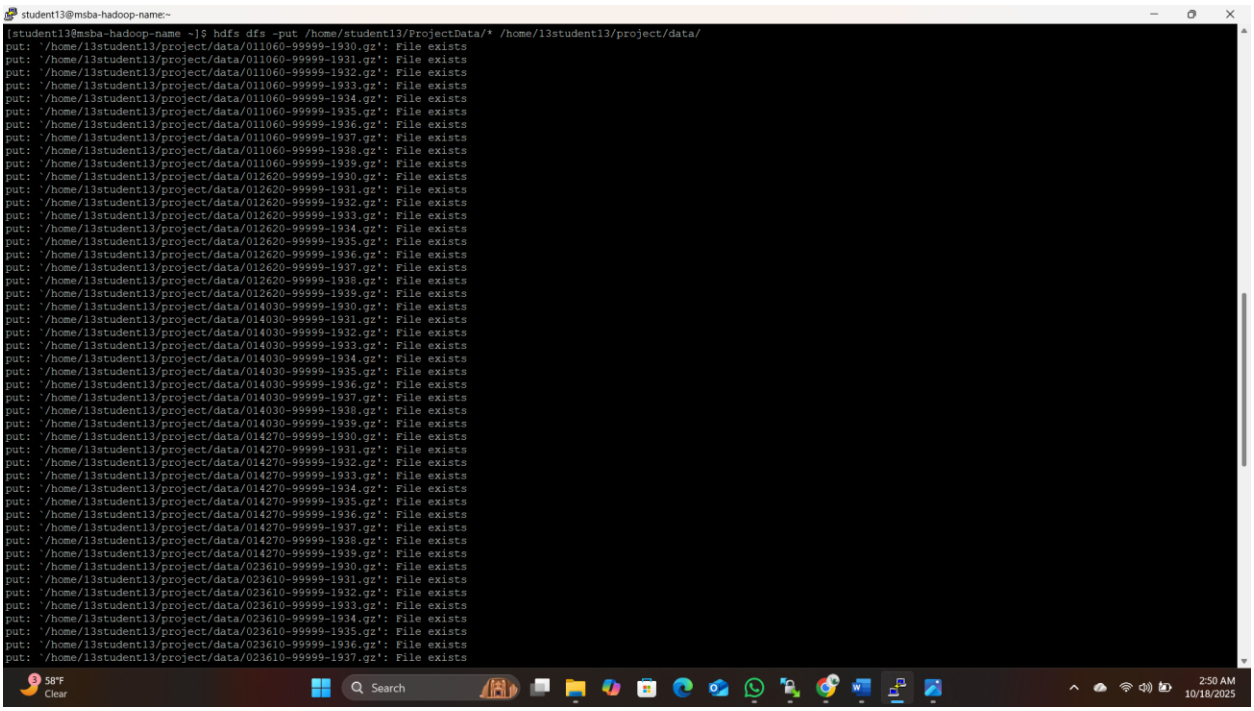
Make hdfs directory to store project data –

hdfs dfs -mkdir -p /home/13student13/project/data



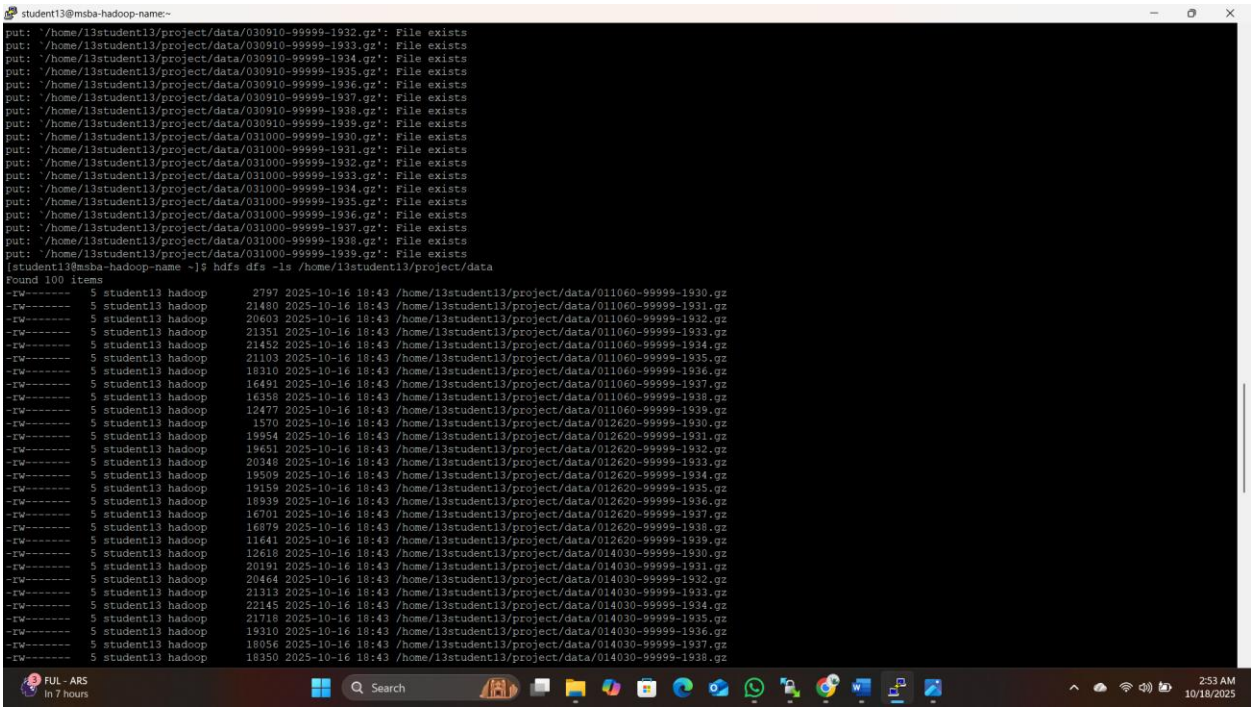
Put ProjectData files in hdfs directory –

hdfs dfs -put /home/student13/ProjectData/* /home/13student13/project/data/



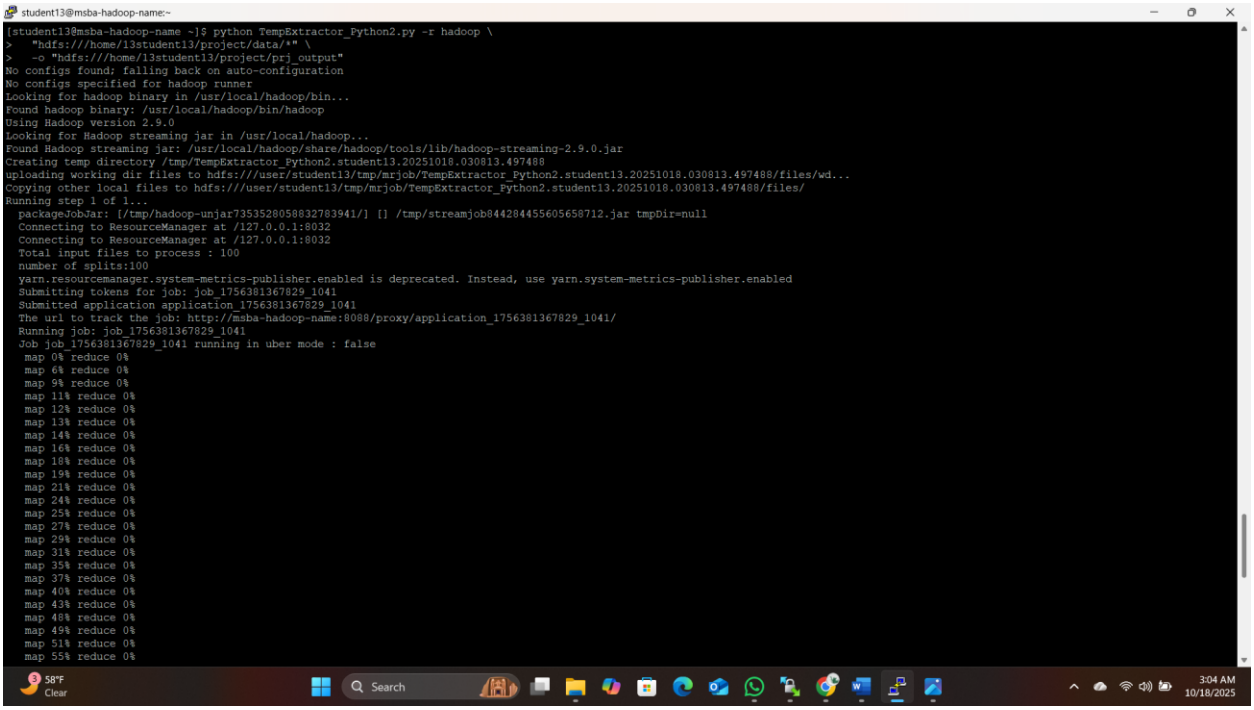
Display data files in hdfs –

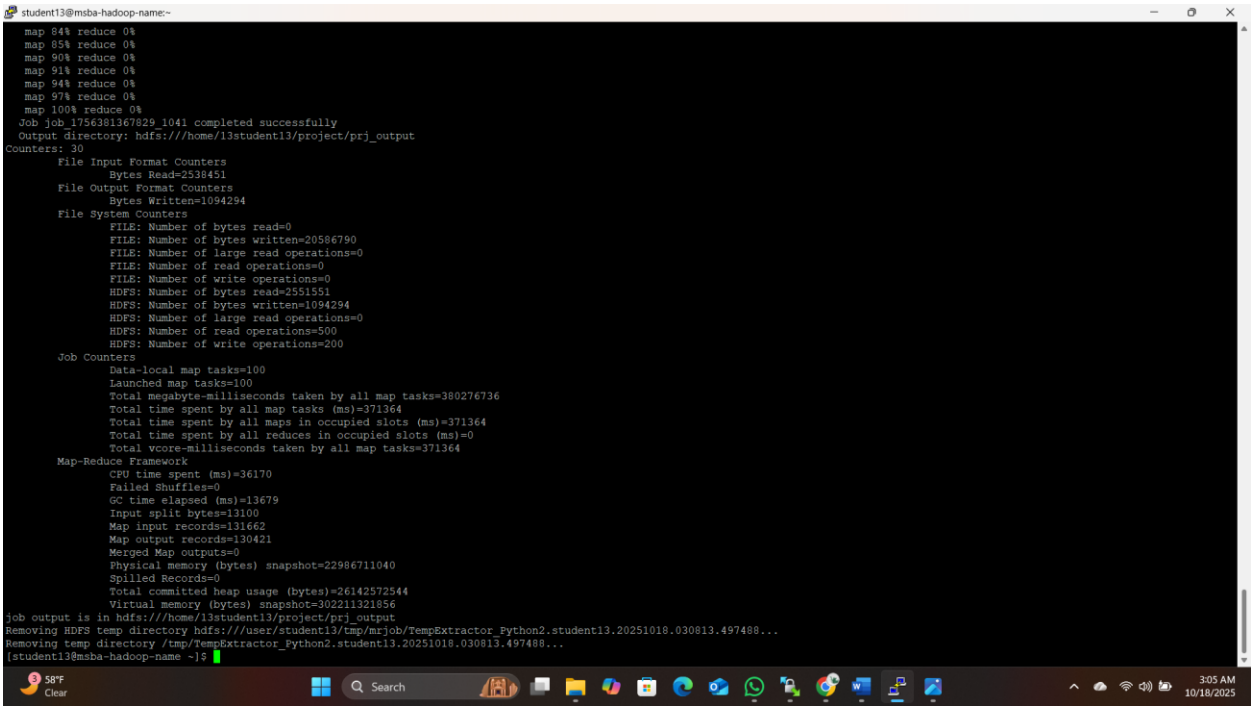
hdfs dfs -ls /home/13student13/project/data



Step 2: Running the file –

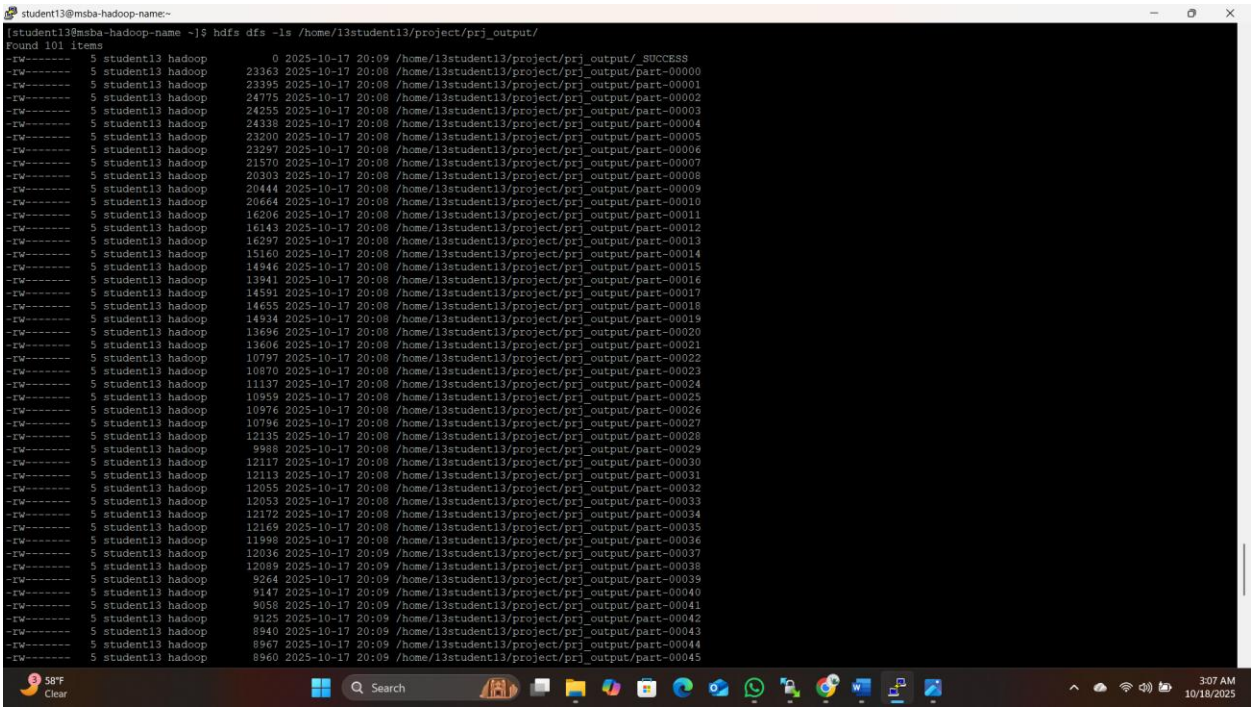
python TempExtractor_Python2.py -r hadoop hdfs:///home/13student13/project/data/* -o hdfs:///home/13student13/project/prj_output/





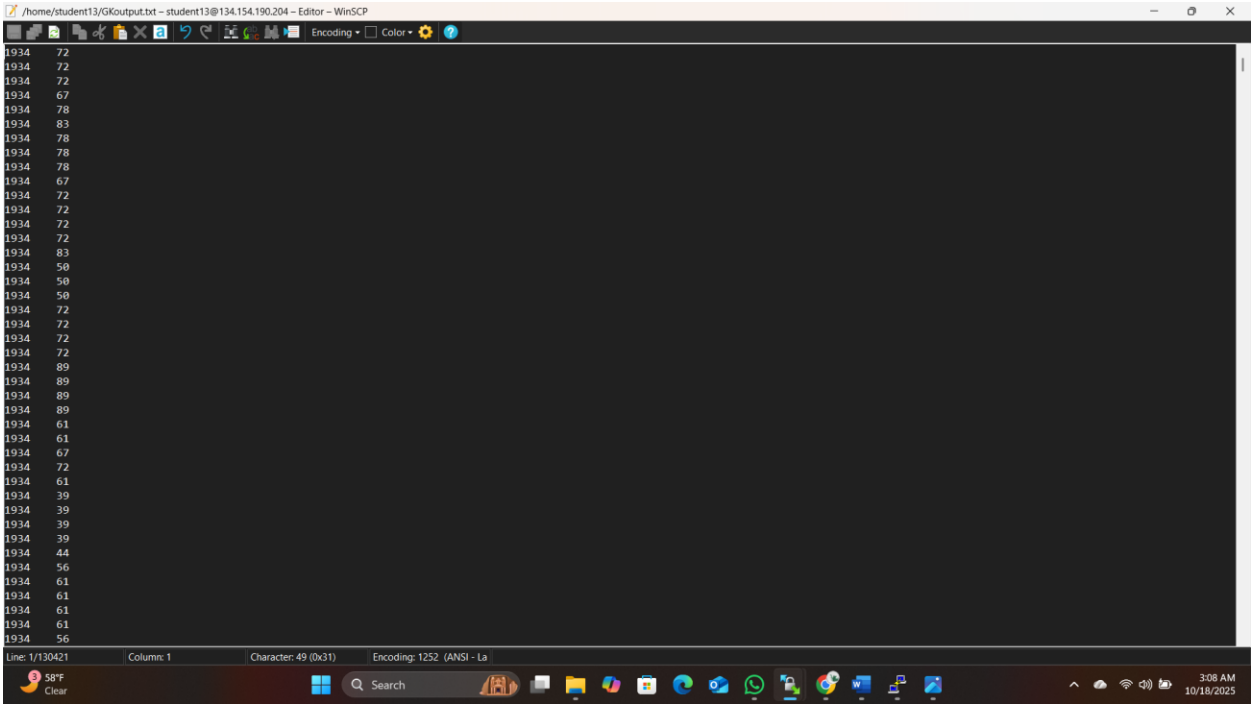
Display Output files which were produced –

hdfs dfs -ls /home/13student13/project/prj_output/



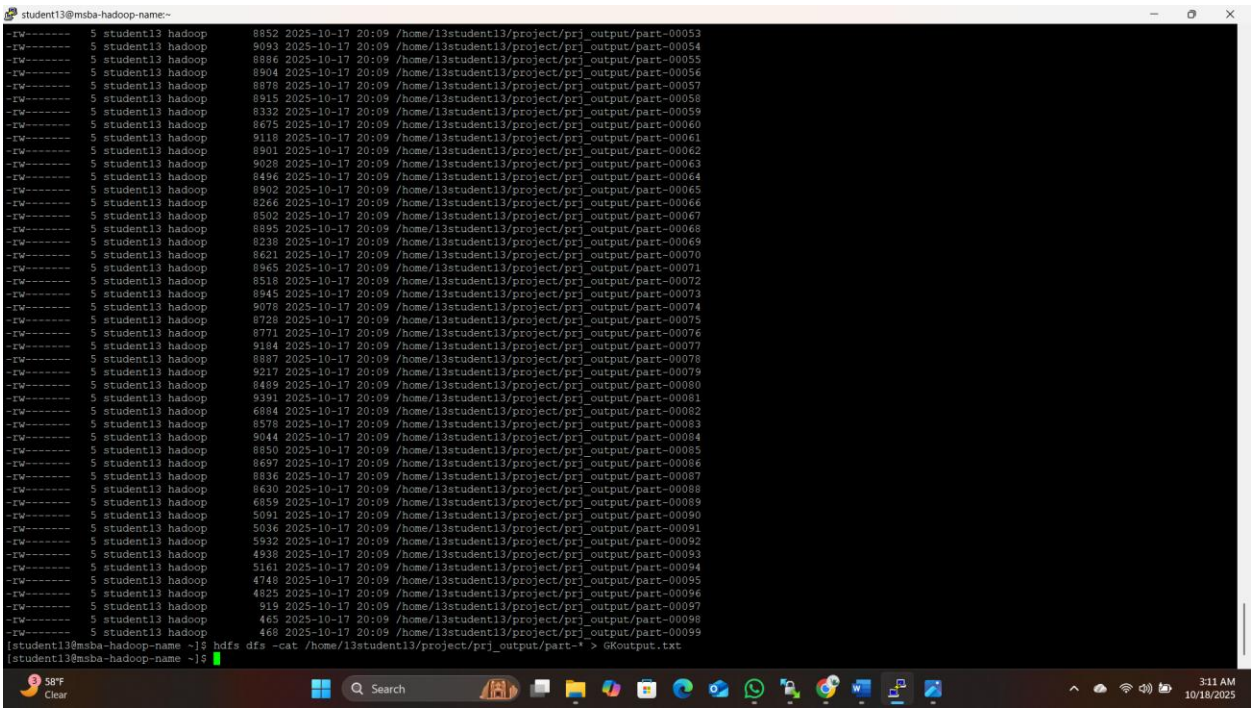
Step 3: Display contents of the text file created –

GKoutput.txt

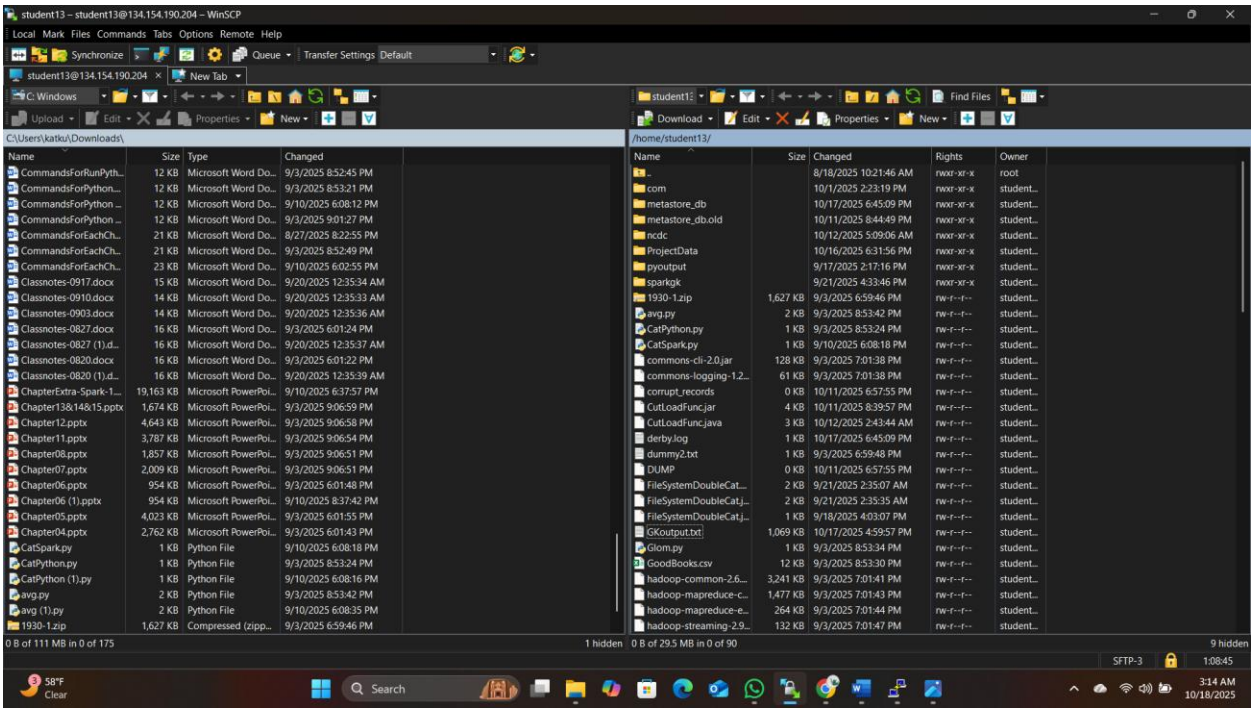


Step 4: Combine all output files and create a merged file GKoutput.txt (containing the results of all output files) in local filesystem

hdfs dfs -cat /home/13student13/project/prj_output/part-* > GKoutput.txt



Created GKoutput.txt file in local filesystem –

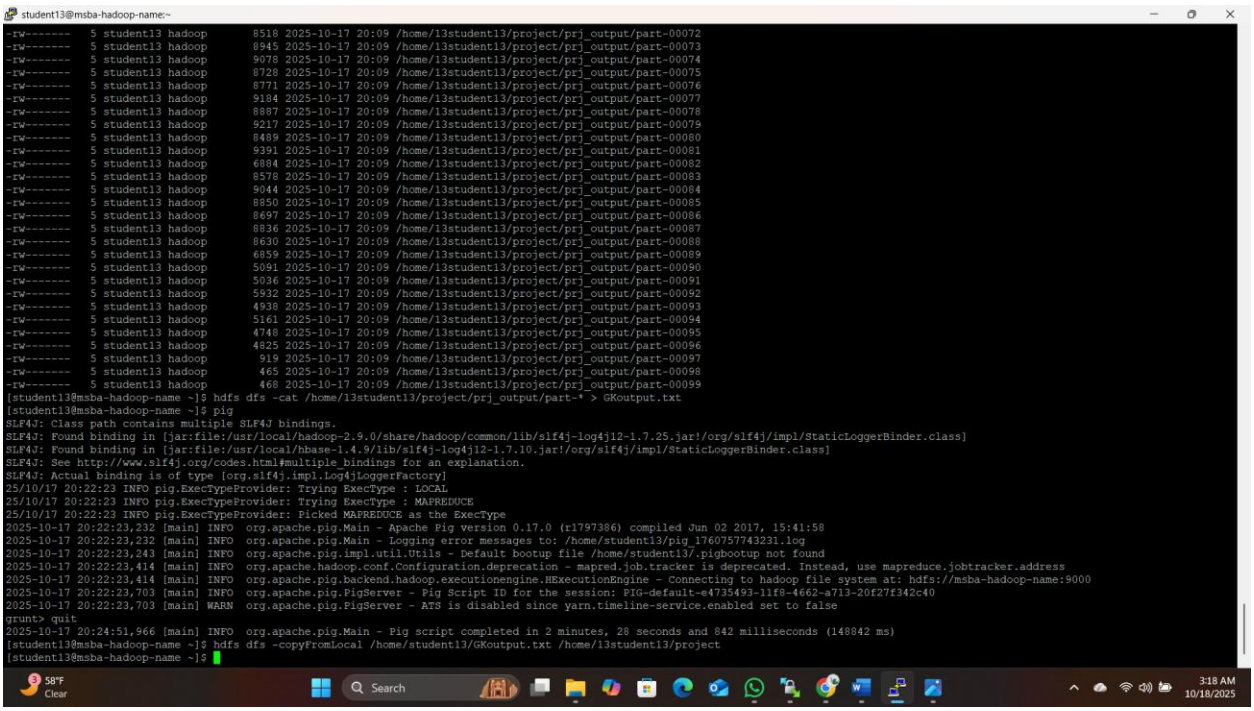


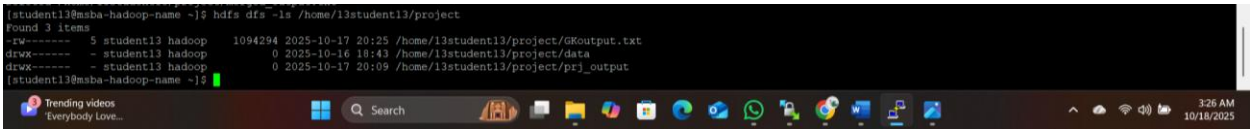
Step 5: Pig commands execution and output

Copy GKoutput.txt from local filesystem to HDFS and display in HDFS –

hdfs dfs -copyFromLocal /home/student13/GKoutput.txt /home/13student13/project

hdfs dfs -ls /home/13student13/project





Using previously produced GKoutput.txt as input for pig LOAD command –

grunt> all_records = LOAD '/home/13student13/project/GKoutput.txt'

>> AS (year:chararray, temperature:int);

Group records by year –

grunt> grouped_records = GROUP all_records BY year;

```
[student13@msba-hadoop-name ~]$ hdfs dfs -ls /home/13student13/project
Found 3 items
-rw-------   5 student13 hadoop    1094294 2025-10-17 20:25 /home/13student13/project/GKoutput.txt
drwx------   - student13 hadoop          0 2025-10-16 18:43 /home/13student13/project/data
drwx------   - student13 hadoop          0 2025-10-17 20:09 /home/13student13/project/prj_output
[student13@msba-hadoop-name ~]$ ^C
[student13@msba-hadoop-name ~]$ pig
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop-2.9.0/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hbase-1.4.9/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
25/10/17 20:35:14 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
25/10/17 20:35:14 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
25/10/17 20:35:14 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2025-10-17 20:35:14,523 [main] INFO  org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2025-10-17 20:35:14,523 [main] INFO  org.apache.pig.Main - Logging error messages to: /home/student13/pig_1760758514522.log
2025-10-17 20:35:14,534 [main] INFO  org.apache.pig.impl.util.Utils - Default bootup file /home/student13/.pigbootup not found
2025-10-17 20:35:14,702 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2025-10-17 20:35:14,702 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://msba-hadoop-name:9000
2025-10-17 20:35:15,004 [main] INFO  org.apache.pig.PigServer - Pig Script ID for the session: PIG-default-163290cd-1b0e-4ab7-a837-b47860b9395a
2025-10-17 20:35:15,007 [main] WARN  org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
grunt> all_records = LOAD '/home/13student13/project/GKoutput.txt'
>> AS (year:chararray, temperature:int);
grunt> grouped_records = GROUP all_records BY year;
grunt> max_min_temp = FOREACH grouped_records GENERATE group AS year
```

For each year, calculate the max and min temperature from all records in that year –

grunt> max_min_temp = FOREACH grouped_records GENERATE group AS year, MAX(all_records.temperature) AS max_temp, MIN(all_records.temperature) AS min_temp;

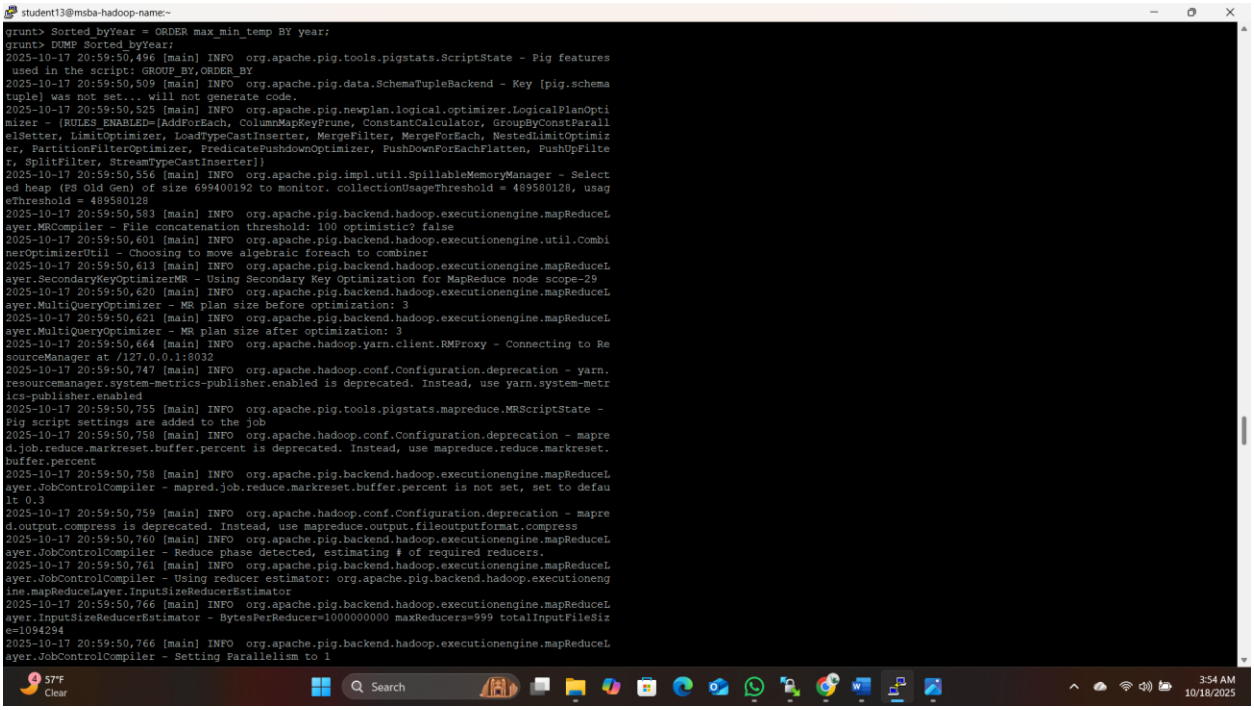
Sort by year so that they appear in ascending order –

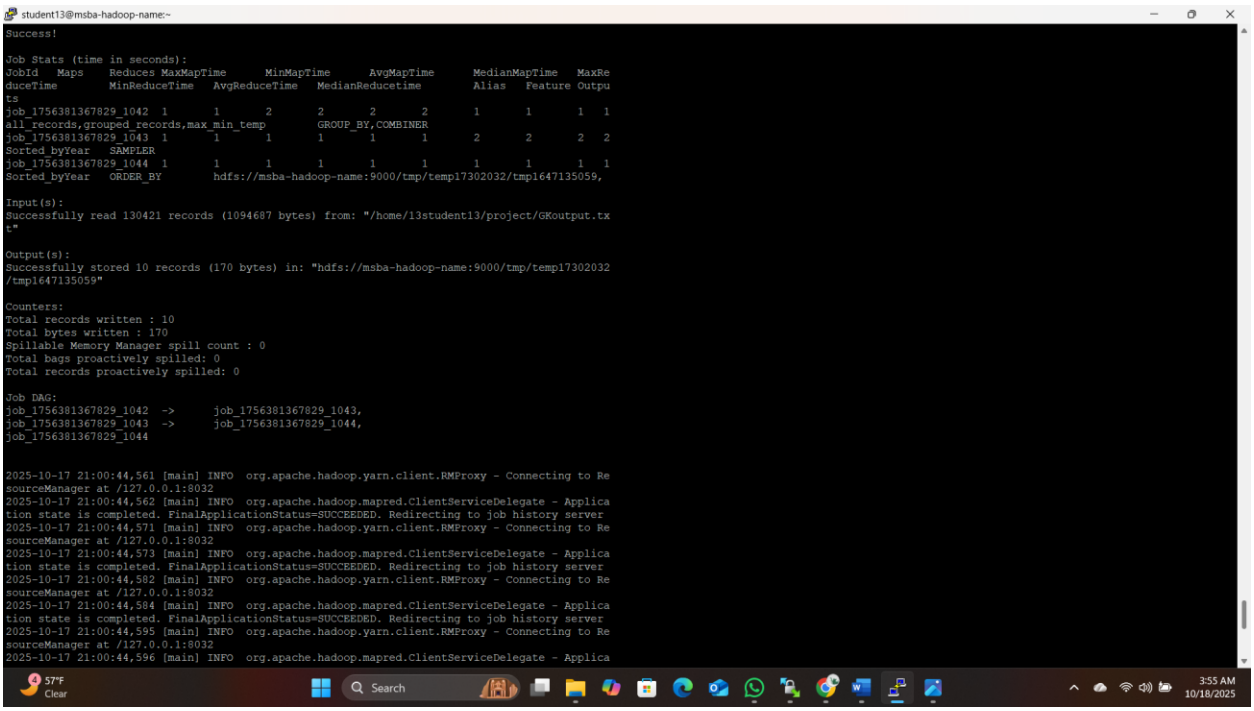
grunt> Sorted_byYear = ORDER max_min_temp BY year;

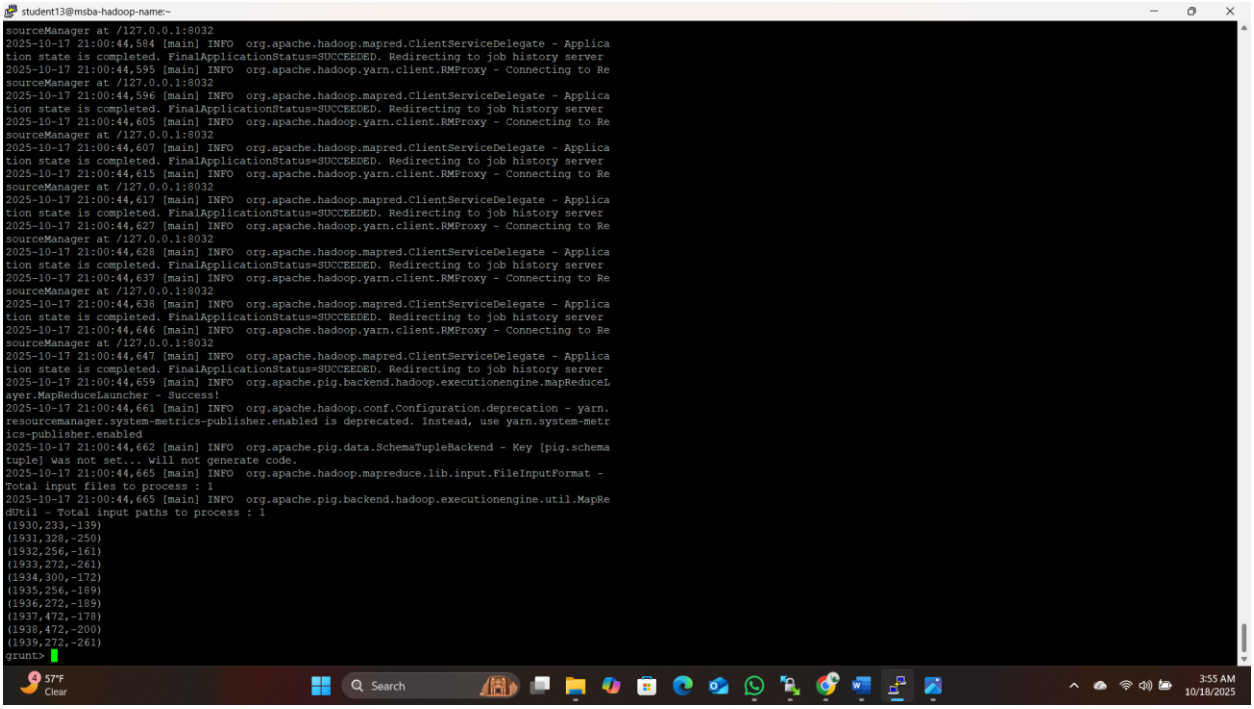


Display the results –

grunt> DUMP Sorted_byYear;







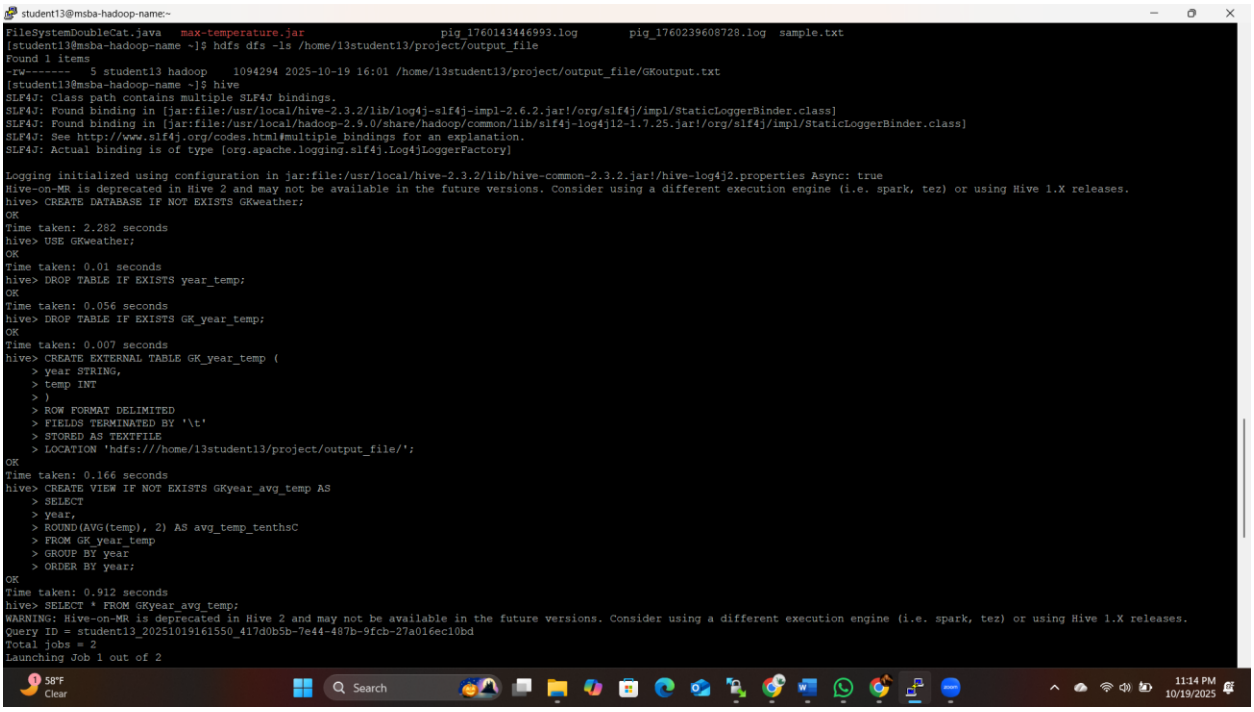
Step 6:

Hive commands execution and output Create DB –

CREATE DATABASE IF NOT EXISTS GKweather;

USE GKweather;

DROP TABLE IF EXISTS GK_year_temp;



Create Table –

CREATE EXTERNAL TABLE GK_year_temp (

year STRING,

temp INT

)

ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'

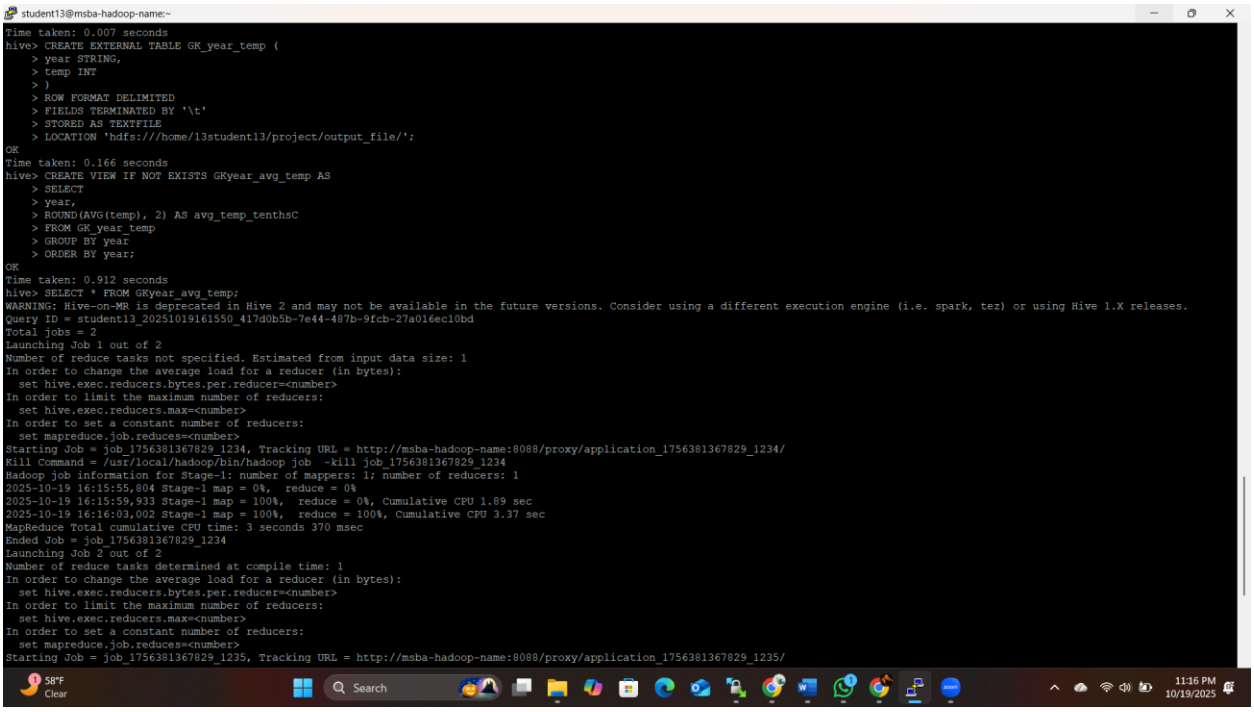
STORED AS TEXTFILE

LOCATION 'hdfs:///home/13student13/project/output_file/';

^(PS created new “output_file” folder on hdfs and copied the GKoutput.txt file in it which is used as input in above command)

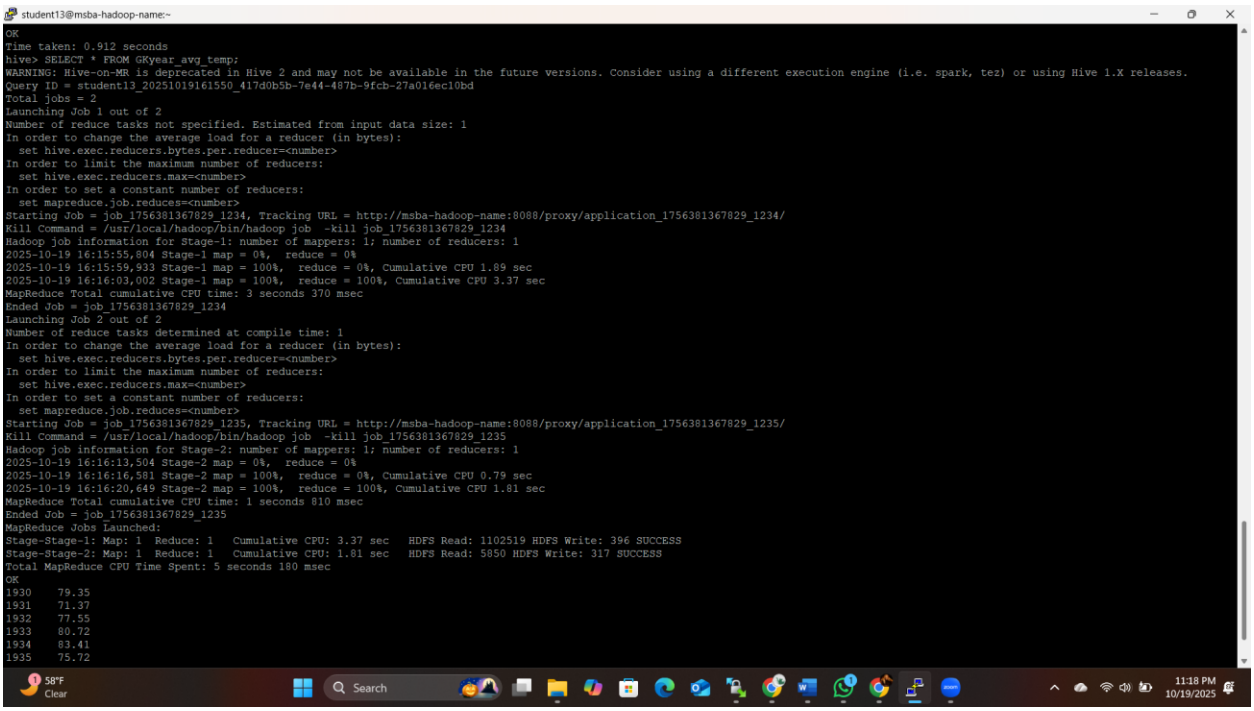
Create View for Avg Temp –

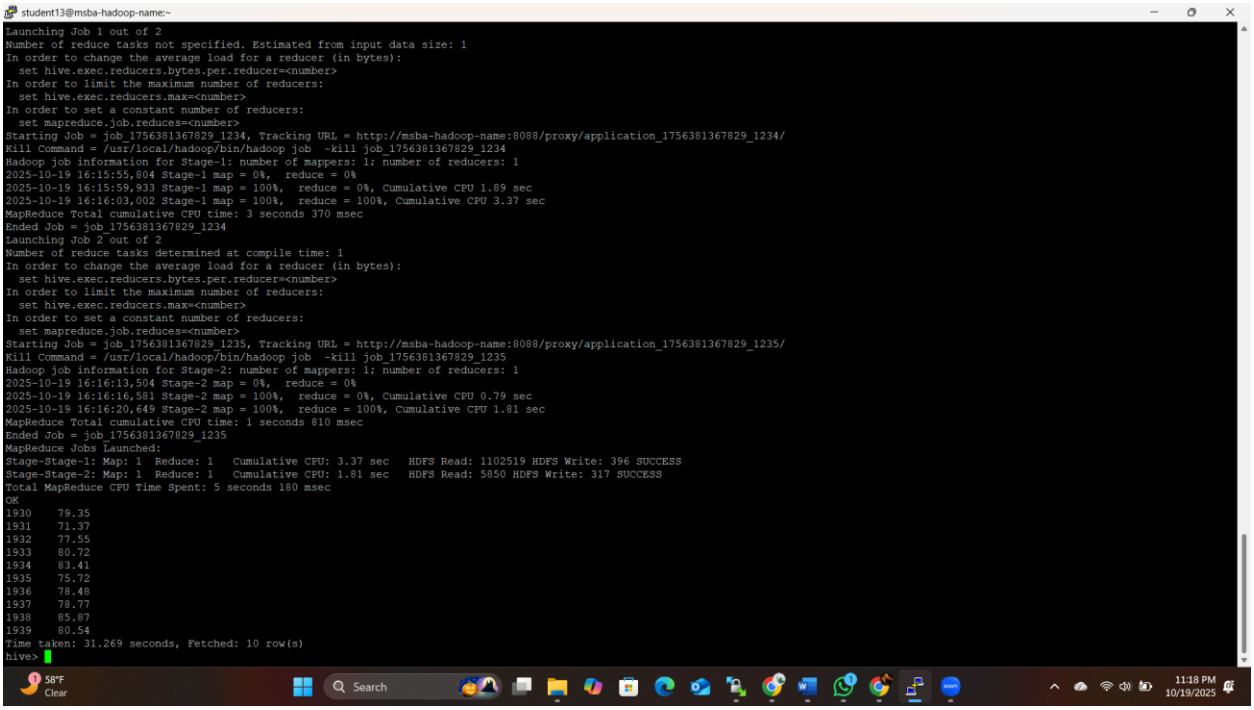
```
CREATE VIEW IF NOT EXISTS GKyear_avg_temp AS
SELECT
year,
ROUND(AVG(temp), 2) AS avg_temp_tenthsC
FROM GK_year_temp
GROUP BY year
ORDER BY year;
```



Display results –

SELECT * FROM GKyear_avg_temp;


student13@msba-hadoop-name:~
OK
Time taken: 0.912 seconds
hive> SELECT * FROM GKyear_avg_temp;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = student13_20251019161550_417d0b5b-7e44-487b-9fcb-27a016ec10bd
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Starting Job = job_1756381367829_1234, Tracking URL = http://msba-hadoop-name:8088/proxy/application_1756381367829_1234/
Kill Command = /usr/local/hadoop/bin/hadoop job -kill job_1756381367829_1234
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2025-10-19 16:15:55,804 Stage-1 map = 0%, reduce = 0%
2025-10-19 16:15:59,933 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.89 sec
2025-10-19 16:16:03,002 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 3.37 sec
MapReduce Total cumulative CPU time: 3 seconds 370 msec
Ended Job = job_1756381367829_1234
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Starting Job = job_1756381367829_1235, Tracking URL = http://msba-hadoop-name:8088/proxy/application_1756381367829_1235/
Kill Command = /usr/local/hadoop/bin/hadoop job -kill job_1756381367829_1235
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2025-10-19 16:16:13,504 Stage-2 map = 0%, reduce = 0%
2025-10-19 16:16:16,581 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 0.79 sec
2025-10-19 16:16:20,649 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 1.81 sec
MapReduce Total cumulative CPU time: 1 seconds 810 msec
Ended Job = job_1756381367829_1235
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.37 sec HDFS Read: 1102519 HDFS Write: 396 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 1.81 sec HDFS Read: 5850 HDFS Write: 317 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 180 msec
OK
1930 79.35
1931 71.37
1932 77.55
1933 80.72
1934 83.41
1935 75.72
58°F Clear
Search
11:18 PM 10/19/2025


student13@msba-hadoop-name:~
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Starting Job = job_1756381367829_1234, Tracking URL = http://msba-hadoop-name:8088/proxy/application_1756381367829_1234/
Kill Command = /usr/local/hadoop/bin/hadoop job -kill job_1756381367829_1234
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2025-10-19 16:15:55,804 Stage-1 map = 0%, reduce = 0%
2025-10-19 16:15:59,933 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.89 sec
2025-10-19 16:16:03,002 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 3.37 sec
MapReduce Total cumulative CPU time: 3 seconds 370 msec
Ended Job = job_1756381367829_1234
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Starting Job = job_1756381367829_1235, Tracking URL = http://msba-hadoop-name:8088/proxy/application_1756381367829_1235/
Kill Command = /usr/local/hadoop/bin/hadoop job -kill job_1756381367829_1235
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2025-10-19 16:16:13,504 Stage-2 map = 0%, reduce = 0%
2025-10-19 16:16:16,581 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 0.79 sec
2025-10-19 16:16:20,649 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 1.81 sec
MapReduce Total cumulative CPU time: 1 seconds 810 msec
Ended Job = job_1756381367829_1235
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.37 sec HDFS Read: 1102519 HDFS Write: 396 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 1.81 sec HDFS Read: 5850 HDFS Write: 317 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 180 msec
OK
1930 79.35
1931 71.37
1932 77.55
1933 80.72
1934 83.41
1935 75.72
1936 78.48
1937 78.77
1938 85.87
1939 80.54
Time taken: 31.269 seconds, Fetched: 10 row(s)
hive>
58°F Clear
Search
11:18 PM 10/19/2025